

Password manager

Александър Колев

ЗМ10700417

Юни 2026

Анализ на заданието.

Заданието изисква да се направи конзолно приложение за менижиране на пароли. Изискани са някои основни команди, които потребителя да може да използва, но за улеснение реших да добавя още, които ще опиша по-долу. Приложението е с цел да складира списък от криптирани пароли, заедно с инструкции за декриптирането им, във файл, защитен с парола. Поддържат се 4 шифъра + 1 тестов, който е недостъпен за потребителя, но все пак не съм изтрил имплементацията му. Шифрите, както и имплементацията им, също ще опиша по-долу. Те биват използвани за криптирането на всяка парола поотделно, а самият файл се защитава с прост XOR шифър.

Архитектура

Приложението съм разделил на 5 части - *core*, *encrypt*, *gui*, *math*, *utils*, като те могат да се разглеждат като 3 основни слоя (*core*, *encrypt*, *gui*) и 2 помощни "библиотеки" (*math*, *utils*).

`clib::Text` и `clib::darray`

Започвам от помощните класове, тъй като те основно менижират сурова памет в приложението, и се използват в абсолютно целият проект.

`clib::darray`

Това е стандартна имплементация на динамичен масив, която поддържа всевъзможни типове данни. Интересното е, че менижирането на паметта се случва посредством `::operator new` и `::operator delete`, вместо с `new T[]` и `delete[]` `T`. Тези оператори просто заделят памет, без да създават `T`-обекти и това ни позволява да имаме динамичен масив от обекти, чиито `default` конструктор е изтрил. Другото интересно за имплементацията е, че изисква от ползвачите на библиотеката да използват само типове с `noexcept move` операции. Това е наложено с коментар, тъй като не успях да открия начин да го наложим без `std::move_if_noexcept`, а използването на копиране навсякъде би забавило програмата.

`clib::Text`

Това е абстракция над `clib::darray`, която НЕ се грижи за памет. Предоставя стандартни `string` функционалности, като четене от `std::istream`, конкатениране, `slice`, всевъзможни конструктори от различни типове (`int`, `char*`) както и съответни +

оператори за конкатенация. Специфичното на имплементацията е, че поддържа в края на "string"-а нул терминатор. Той не се използва по никакъв начин в изчисляването на дължината на стринга, или за логиката, свързана с имплементацията на `clib::Text` методите.. Причината за '\0' е възможността за създаване на `Text::raw()` const метод, който връща нормален "c string", който може да се използва за работа с stl функции, например

```
void PasswordFile::load(const clib::Text& password) {
    std::ifstream file(path.raw(), std::ios::binary);
    ....
}
```

което е много удобно.

math

VignereCipher и HillCipher, за които в детайл ще говорим по-долу, използват аритметика в пръстен на остатъците Z/nZ и/или работа с матрици (изчисляване на обратна, умножение). За тази цел са дефинирани 2 класа

`RemRing<int Mod>` Шаблонен клас, който имплементира всички нужни аритметични операции за работа в пръстена $Z/ModZ$ и две негови инстанции

```
using Z26 = RemRing<26>;
using Z96 = RemRing<96>;
```

тъй като в HillCipher работим с малките букви от азбуката, а в VignereCipher работим с всички четими букви от ASCII таблицата.

SqMatrix

В имплементацията съм описал, че това е клас специфично за работа с Z26 квадратни матрици, което го прави особено неабстрактен, но тъй като това е единствения тип матрици, който се използва, тази имплементация е напълно достатъчна. Особеното е, че матрицата се конструира от тип `clib::darray<clib::darray<Z26>>`, като конструктора хвърля грешка, ако матрицата е необратима. Обратна се пресмята веднъж, при задаване от потребителя, след което се пази във файла, заедно с необратната. `SqMatrix` знае как да се сериализира и създава от прочетен от файл низ.

gui

Това е юзър-фейсинг частта на приложението. В `gui` се дефинира `gui::Shell` класът, който задава абстракция над `std::cin` и `std::cout` и менижира изпълнението на потребителски команди. Тук използвам функции от по-висок ред, което не е чисто ООП, но улеснява имплементацията на регистър с команди. `gui::Shell` съхранява в себе си масив от тип `gui::Command`, където командата е проста структура

```
struct Command {
    clib::Text command;
```

```

        clib::Text usage;
        clib::Text description;
        void (*call)(Command& cmd, Shell& shell, const clib::darray<clib::Text>&
t);
    }

```

от този вид. Масивът се създава в конструктура на gui::Shell. Ползата от тази идея, е че добавянето на нови команди изисква променянето на кода на едно единствено място - конструктура на gui::Shell. Ето ще дам пример как сме имплементирали командата load

```

        commands.add(Command(
            "load", // Изписване на самата команда
            "load <website> [<username>]", // Изискани полета
            "Prints on screen all password entries that match your query with
passwords DECRYPTED.", //Описание
            [](Command& cmd, Shell& shell, const clib::darray<clib::Text>& t) {
//Имплементация
                if(!shell.pm.hasOpenFile()) {
                    shell.print_line("No file open");
                    return;
                }
                //... самото изпълнение
            }
        ));

```

Този gui::Shell клас ни позволява изключително проста и лесна имплементация на цялата потребителска част. Самото изпълнение на командите се случва по следния начин

```

gui::Shell shell;

clib::Text line;
shell.print_line("Welcome to password manager. If you are new type \"help\"
for a list of commands.");
while(1) {

    if(!shell.isRunning()) {
        shell.print_line("Goodbye!");
        return 0;
    }
    clib::getline(shell.in(), line);

    clib::darray<clib::Text> tokens = line.trim().split(' ');

    if(tokens.empty()) continue;

    clib::Text cmd = tokens[0];
    tokens.remove(0);

```

```

    shell.execute(cmd, tokens);
}

```

инструкции на потребителя се подават така

```

commands.add(Command(
    "help",
    "help",
    "How did we get here?",
    [] (Command&, Shell& shell, const clib::darray<clib::Text>&) {
        for(auto& cmd: shell.commands) {
            shell.print_line(cmd.usage + " - " + cmd.description);
        }
    }
));

```

И една бележка към самото изпълнение - Без <function> хедъра не намерих начин да ламбда функциите да виждат променливи. Поради тази причина всичките изискват референция към себе си и към *this под формата на Shell& shell. Алтернативата на това би била идея, подобна на тази, която съм използвал за CipherFactory, но това би изисквало създаването на n малки класа, за n - броят на командите в приложението, което в момента се избягва. Сегашната имплементация позволява добавяне на команди чрез промяна на кода в единствен файл.

Имплементацията, както е по изискване, поддържа страниране, посредством

```

template<typename T>
void paginate(const clib::darray<T>& items, void (*printItem)(Shell&,
const T&));

```

която пак има същия ламбда проблем, за това изисква Shell&

Приложението изисква на потребителя парола за файл всеки път, когато го запазва. Това е с цел да се избегне кеширане на паролата в паметта, докато потребителя използва приложението.

core

Тук се извършва работата с файлове, криптирането и декриптирането на файлове и пароли.

Password.h и PasswordView.h

Тъй като по изискване load командата не достъпва "поверителната" информация за всяко Entry, съм създал и PasswordView който е надклас на PasswordEntry с цел той да бъде предоставян на gui слоят от приложението.

```

class PasswordView {
    clib::Text website;
    clib::Text username;
};

```

```
class PasswordEntry: public PasswordView {
    clib::Text passwordEncrypted;
    encrypt::Cipher* cipher;
};
```

Entry-тата не отговарят за своето криптиране-разкриптиране, те са общо взето прости структури, които знаят как да се сериализират, за да бъдат записани във файл и знаят и шифъра с който са криптирани. Тъй като шифрите са полиморфни инстанции, те се съхраняват като указатели, като всяка парола отговаря за паметта, заделена за нейния шифър. По изискване всички пароли могат да са с различни шифри, съответно PasswordEntry е най-коректното място тази памет да се притежава.

PasswordFile.h PasswordFile може да се опише като абстракция File + PasswordEntry[]. Той отговаря за складирането на списък от PasswordEntry във криптиран файл, за самото разкриптиране на паролите, и за предоставянето им на потребителя. Класът поддържа нужните CRUD операции, заедно с доста гъвкави методи за търсене и филтриране на списъла с пароли. Освен това отговаря и за самото сериализиране/десериализиране на данните, за криптирането и четенето на файла, както и за четене и писане в него. Тъй като това е доста отговорност за един клас, сериализаренто е дефинирано в *Serialize.h*, а криптирането на файла в *FileEncrypt.h*. PasswordFile се грижи за паметта, заделена за шифъра по подразбиране, като на всяко PasswordEntry се дава копие, посредством `encrypt::Cipher::clone`.

PasswordManager.h

Мениджъра дефакто е връзката между потребителя и PasswordFile. Той отговаря за отварянето и затварянето на файл за пароли, както и за извикване на подходящите CRUD заявки, зададени от потребителя, посредством `gui::Shell```. Приложението поддържа един отворен файл във всеки един момент, като PasswordManager`` се грижи точно за това.

encrypt

Програмата, както се изисква, поддържа HillCipher, TextCode, VignereCipher и CaesarCipher. Тези методи за шифриране са имплементирани чрез един абстрактен клас `encrypt::Cipher` и клас `encrypt::CipherFactory`. Cipher задава основния шаблон за един шифър, който включва методи за криптиране, декриптиране, сериализиране и копиране (посредством `clone()` const метод).

```
class Cipher {
public:
    virtual clib::Text encrypt(const clib::Text& text) const = 0;
    virtual clib::Text decrypt(const clib::Text& text) const = 0;

    virtual clib::Text serialize() const = 0;
    virtual clib::Text type() const = 0;

    virtual ~Cipher() = default;
```

```

    virtual Cipher* clone() const = 0;
};

```

CipherFactory изисква от програмиста да зададе 2 начина за създаване на шифър - чрез списък от низове, прочетен от файл, или през gui::Shell обект.

```

class CipherFactory {
public:
    virtual Cipher* fromArgs(const clib::darray<clib::Text>& args) const = 0;
    virtual Cipher* fromShell(gui::Shell& shell) const = 0;
    virtual clib::Text type() const = 0;

    virtual ~CipherFactory() = default;
};

```

Сега чрез тези класове от програмиста се изисква, при създаване на нов шифър, само да ги наследи и да даде имплементации за нужните методи. Ето един прост пример с TestCipher.

```

class TestCipher: public Cipher {
public:
    TestCipher(const clib::Text& secretword);
    clib::Text encrypt(const clib::Text& text) const override;
    clib::Text decrypt(const clib::Text& text) const override;
    TestCipher* clone() const override;
    clib::Text type() const override;
    clib::Text serialize() const override;
    static clib::Text fromPassword(const clib::Text& password);
private:
    clib::Text secretword;
};

class TestCipherFactory: public CipherFactory {
public:
    TestCipher* fromArgs(const clib::darray<clib::Text>& args) const
override;
    TestCipher* fromShell(gui::Shell& shell) const override;
    clib::Text type() const override;
};

```

и след това просто да го добави към регистъра CipherRegistry

```

CipherRegistry::CipherRegistry() {
    //add(new TestCipherFactory());
    add(new HillCipherFactory());
    add(new CaesarCipherFactory());
    add(new VignereCipherFactory());
    add(new TextCodeFactory());
}

```

Ако това бъде изпълнено вече шифъра може да се използва навсякъде в програмата без никакви лимитации, което покрива изискването добавянето на нови шифри да става лесно.

Имплементация на шифри

Hill Cipher по подразбиране работи само върху думи с дължина n за $n \times n$ ключ-матрица. В имплементацията на приложението той работи с думи с дължина kn за $n \times n$ ключ-матрица и k цяло число, което е направено като думата се разделя на части с дължина n , като всяка от тях се криптира/декриптира поотделно. В идеи за развитие на приложението е да се направи да работи за думи с произволна дължина.

Шифриране на файл

За криптирането на файла с паролите съм създал проста имплементация на стандартно XOR криптиране чрез ключ.

<https://www.geeksforgeeks.org/dsa/xor-cipher/>

Допълнителни команди

help

Изкарва на екрана списък с всички дефинирани команди, тяхното описание и начинът им на ползване.

save_file

Запазва отвореният файл, като преди това изисква паролата на потребителя

ciphers

Изкарва на екрана списък с всички регистрирани шифри

clear

Изчиства екрана. Командата използва ANSI кодове, съответно работи само в терминали, които ги поддържат.

https://en.wikipedia.org/wiki/ANSI_escape_code

Грешки

В програмата има дефинирани няколко специални грешки, с цел да улеснят работата с файлове, тъй като нещо може да се обърка по много начини - Проблем със самия файл, проблем с декриптирането му, с десериализирането на самия файл, с разчитане на ключа за криптиране на някоя парола, необратима матрица, необратим клас-остатък и други.

```
class FileError : public std::runtime_error
class PasswordModifyError : public std::runtime_error
class InvalidCipherTypeError : public EncryptionError
```

```
class MathError : public std::invalid_argument
class NoFileOpenError : public std::runtime_error
class EncryptionError : public std::runtime_error
```

Тестване

Шифрите и работата с файлове се тества периодично чрез void test() функцията в main.cpp, а графичният интерфейс се тества ръчно. Това е номер 1 нещо в бъдещите подобрения - да се изготвят unit тестове за всеки шифър и всяка круд операция, както и за всяка потребителска команда.

Бъдещи подобрения

1. unit тестове за всеки шифър и всяка круд операция, както и за всяка потребителска команда.

2. Списък от ръчни тестове, чрез които да се верифицира че приложението изглежда добре и е ергономично, след всяка нова добавка

3. import/export csv фийчър.

Стандартно е Password Manager приложенияте да имат възможност да експортират паролите си в стандартизиран формат, което е с цел да улесни смяната от един Password Manager към друг. Това е подобрение, което ще направи приложението ми доста по-полезно - експортирането, за да може хората да използват cloud-based мениджър на пароли и импортиране, за да може да не зависиш от cloud-based приложения, и всичко да си поддържаш сам.

4. Подобряване на криптирането.

В момента приложението използва просто XOR криптиране, което не е изобщо безопасно. Усложняване на шифъра, чрез например сол би направило приложението доста по-приемливо за ползване, тъй като в момента не се препоръчва наистина да се складира пароли в него.

5. GUI подобрения

В момента ако потребителя е сбъркал една буква в дългата команда, която се опитва да напише, трябва да я пише наново - не може със "стрелка нагоре" да се върне, да поправи грешката си, и да продължи работа. Това би улеснило работата с приложението доста.

6. Запазване на файл (Бъг)

В момента се изисква от потребителя парола всеки път като иска да запази файла, което не е лошо. Проблемът е, че няма как да знаем дали потребителя използва същата парола за запазване на файл, която е използвал и за създаване. Съответно ако напише password вместо password няма да изникне проблем веднага, но следващия път като се опита да си отвори файла, ще се чуди защо password не става. Решение на този проблем би било да се пазим някакъв хеш на паролата, с която потребителя е отворил файла. Този хеш може спокойно да се

пази в рам паметта, докато се използва приложението, тъй като не може много да се помогне за разбиване на шифъра. Но ни дава възможност да разберем дали паролата за запазване от потребителя съвпада с паролата за отваряне на файл.

Източници

Изчистване на конзолата <https://stackoverflow.com/questions/6486289/how-to-clear-the-console-in-c> https://en.wikipedia.org/wiki/ANSI_escape_code

Идеи за добри практики <https://refactoring.guru/design-patterns/catalog>

Работа с езика <https://cppreference.com/>

Имплементация на `clib::darray` <https://en.cppreference.com/cpp/container/vector>
<https://stackoverflow.com/questions/3536153/c-dynamically-growing-array>

Алгебра за `math::` https://en.wikipedia.org/wiki/Euclidean_algorithm
https://en.wikipedia.org/wiki/Quotient_ring https://en.wikipedia.org/wiki/Matrix_ring

Източници за самите шифри https://en.wikipedia.org/wiki/Vigen%C3%A8re_cipher
https://en.wikipedia.org/wiki/Caesar_cipher https://en.wikipedia.org/wiki/Hill_cipher

Идея за unit тестове <https://www.ibm.com/think/topics/unit-testing>

Идея за csv import-export <https://support.google.com/chrome/answer/13068232>

Нужда от промяна на шифъра <https://arpitbhayani.me/blogs/decipher-repeated-key-xor/>

Сегашния шифър на файла <https://www.geeksforgeeks.org/dsa/xor-cipher/>

Други линкове

Предишна работа с динамични списъци в C/C++ <https://github.com/Aliksalot/libc>

Линк към гитхъба на изпълнението
<https://github.com/Aliksalot/passwordManager/releases>